

Homework 04

Question 1: Signal strength and correlated substitutes

Answer

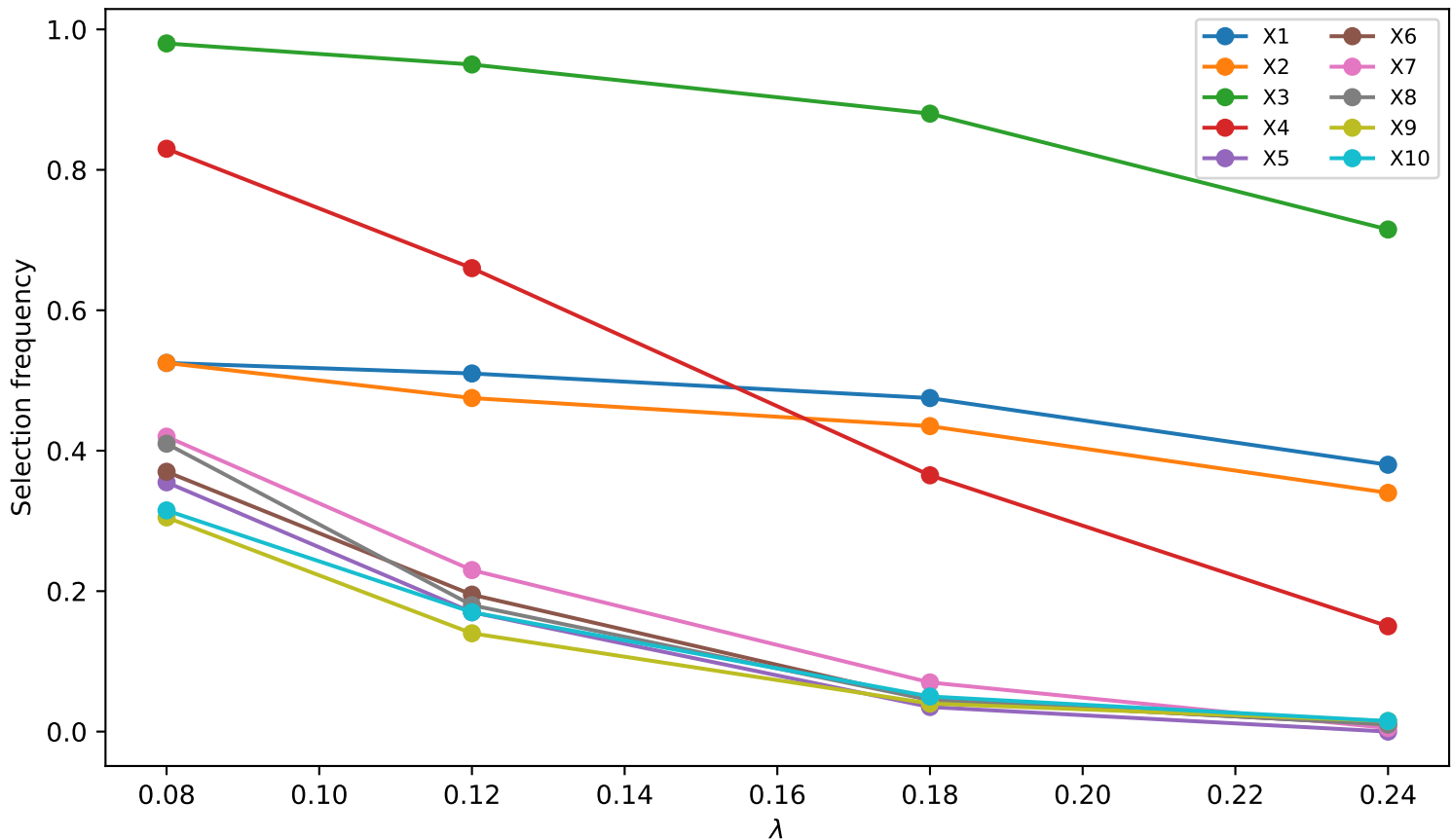
i Note

A note on random-number generation. The simulation below draws with `rng = np.random.default_rng(43246)` (NumPy's modern Generator API) rather than the legacy `np.random.seed() + np.random.randn()` pattern. This matters here specifically because `sklearn.linear_model.Lasso/ElasticNet` call `check_random_state(None)` internally on every `.fit()` call — even under the default `selection="cyclic"` — which silently advances the *legacy* global RNG's internal position. With the legacy pattern, that means the exact sequence of simulated datasets would depend on how many models get fit per repetition, so Question 1 (one lasso fit per repetition/penalty) and Question 2 (lasso *and* elastic net) would silently end up simulating different data even with the same seed — breaking the requirement that both questions “produce the same datasets.” A `default_rng()` generator is a separate, local object that `.fit()` never touches, so this can't happen.

Part a

All 800 fits converged: True

Selection frequency vs. λ , by covariate



Mean absolute fitted slope vs. λ , by covariate

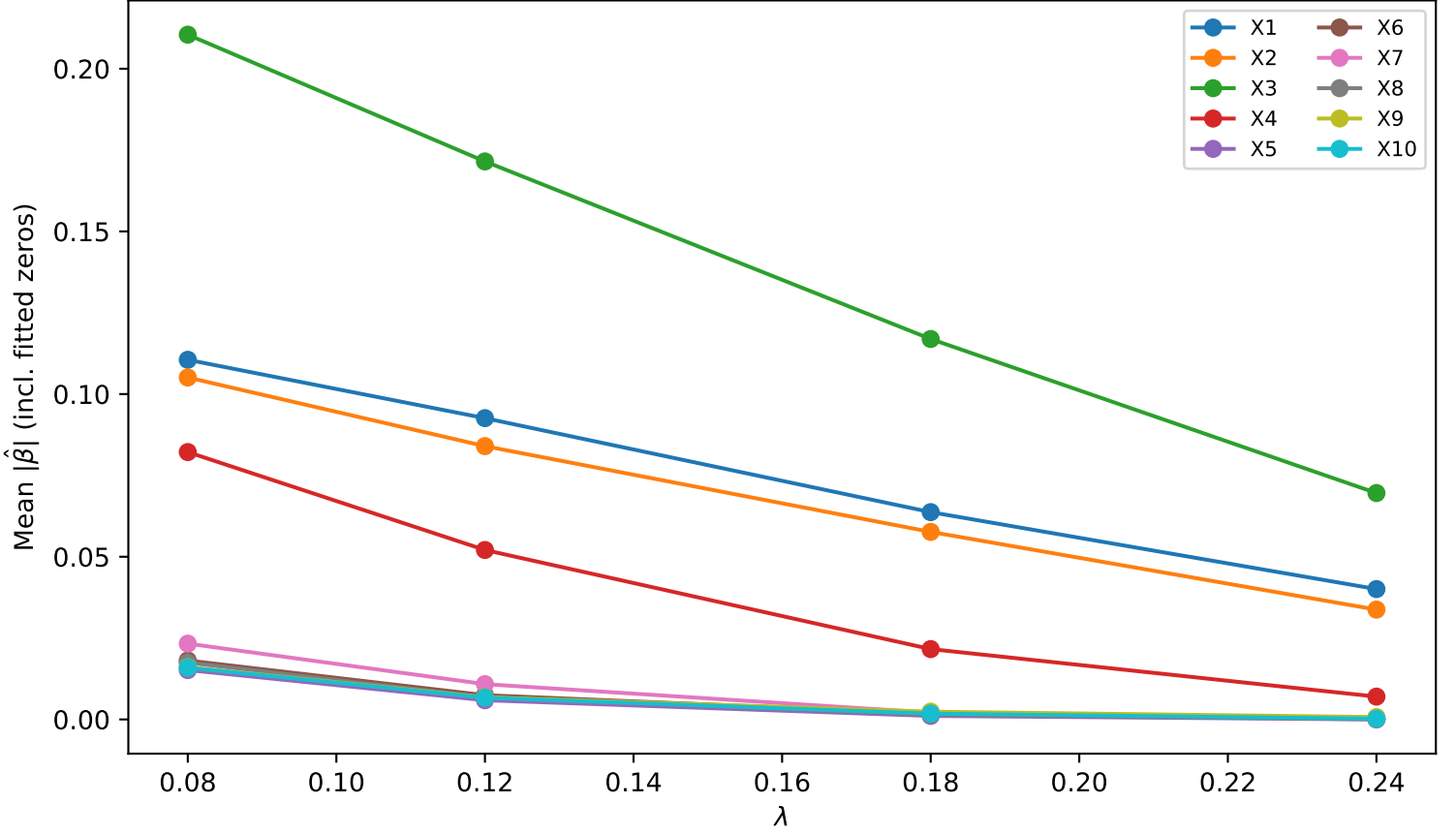


Table 1: Selection frequency by covariate and λ .

	$\lambda=0.08$	$\lambda=0.12$	$\lambda=0.18$	$\lambda=0.24$
X1	0.525	0.510	0.475	0.380
X2	0.525	0.475	0.435	0.340
X3	0.980	0.950	0.880	0.715
X4	0.830	0.660	0.365	0.150
X5	0.355	0.170	0.035	0.000
X6	0.370	0.195	0.045	0.010
X7	0.420	0.230	0.070	0.005
X8	0.410	0.180	0.045	0.010
X9	0.305	0.140	0.040	0.015
X10	0.315	0.170	0.050	0.015

Table 2: Mean $|\hat{\beta}_j|$ (including fitted zeros) by covariate and λ .

	$\lambda=0.08$	$\lambda=0.12$	$\lambda=0.18$	$\lambda=0.24$
X1	0.1105	0.0926	0.0637	0.0401
X2	0.1051	0.0840	0.0576	0.0338
X3	0.2105	0.1715	0.1169	0.0696
X4	0.0822	0.0521	0.0216	0.0070
X5	0.0152	0.0059	0.0011	0.0000
X6	0.0181	0.0075	0.0017	0.0004
X7	0.0233	0.0109	0.0021	0.0002
X8	0.0174	0.0068	0.0015	0.0001
X9	0.0161	0.0068	0.0024	0.0007
X10	0.0159	0.0067	0.0018	0.0002

As λ increases from 0.08 to 0.24, mean $|\hat{\beta}_j|$ falls monotonically for *every* covariate (Table 2, figures above) — the usual ℓ_1 shrinkage/soft-thresholding effect. Selection frequency (Table 1, figures above) falls for every covariate as λ grows, but at very different rates. X_3 (true coefficient 0.30) is selected in essentially every replicate at the smallest penalty (0.980) and remains the most frequently selected covariate throughout, still at 0.715 at $\lambda = 0.24$. X_1 , X_2 , and X_4 (true coefficient 0.15 each) start noticeably lower (0.53–0.83) and fall faster, with X_4 collapsing to 0.150 by $\lambda = 0.24$. The pure noise covariates $X_5, \dots, X_{10}$ (true coefficient 0) start around 0.31–0.42 at $\lambda = 0.08$ — nontrivial, since with $n = 120$ and ten correlated-with-noise draws some spurious selection is expected at a small penalty — but are driven toward zero rapidly, averaging about 0.05 by $\lambda = 0.18$ and under 0.01 by $\lambda = 0.24$. This ordering matches the size of the true coefficient: a larger true signal survives a larger penalty before its estimated effect is shrunk below the point where it gets thresholded to exactly zero.

Part b

$\lambda=0.08$: only_X1=0.465 only_X2=0.465 both=0.060 neither=0.010
 $\lambda=0.12$: only_X1=0.475 only_X2=0.440 both=0.035 neither=0.050
 $\lambda=0.18$: only_X1=0.445 only_X2=0.405 both=0.030 neither=0.120
 $\lambda=0.24$: only_X1=0.350 only_X2=0.310 both=0.030 neither=0.310

Table 3: Joint and marginal selection outcomes for (X_1, X_2) .

	Only X_1	Only X_2	Both	Neither
$\lambda=0.08$	0.465	0.465	0.060	0.01
$\lambda=0.12$	0.475	0.440	0.035	0.05
$\lambda=0.18$	0.445	0.405	0.030	0.12
$\lambda=0.24$	0.350	0.310	0.030	0.31

The marginal selection frequencies of X_1 and X_2 are close at every λ (Table 3) — 0.525 vs. 0.525 at $\lambda = 0.08$, narrowing to 0.380 vs. 0.340 at $\lambda = 0.24$ — consistent with $\text{corr}(X_1, X_2) = 0.999$ and identical true coefficients (0.15 each): the two variables carry essentially the same information about Y , so there is no structural reason for lasso to favor one over the other on average, and the small gap reflects only sampling variation from the specific noise draw Z used to construct X_2 from X_1 in each replicate.

As λ increases, **both selected** declines from 0.060 to 0.030 while **neither selected** rises sharply, from 0.010 at $\lambda = 0.08$ to 0.310 at $\lambda = 0.24$; **only** X_1 and **only** X_2 remain the dominant outcomes at the smaller penalties, together accounting for over 90% of replicates at $\lambda = 0.08$, though that combined share falls to about 66% by $\lambda = 0.24$ as **neither** grows. This is the signature lasso behavior under near-collinearity: because X_1 and X_2 are almost interchangeable predictors, the ℓ_1 penalty makes it cheaper to load the shared effect onto a single member of the pair and zero out the other than to keep two nearly redundant coefficients active simultaneously. Which one “wins” is sensitive to the noise realization in a given replicate, producing a sparse fitted rule that omits one member of the pair even though *both* appear with equal weight (0.15 each) in the true generating equation. As λ grows further, it becomes cheap enough to drop even the single retained variable, so **neither** grows to over 30% of replicates by $\lambda = 0.24$ while **both** falls to half its $\lambda = 0.08$ level.

Part c

Table 4: Selection frequency: X_3 , X_4 , and average over $X_5, \dots, X_{10}$.

	X_3	X_4	$\text{avg}(X_5, \dots, X_{10})$
$\lambda=0.08$	0.980	0.830	0.362
$\lambda=0.12$	0.950	0.660	0.181
$\lambda=0.18$	0.880	0.365	0.048
$\lambda=0.24$	0.715	0.150	0.009

At every λ , $\text{freq}(X_3) > \text{freq}(X_4) \gg \text{avg. freq}(X_5, \dots, X_{10})$ (Table 4): at $\lambda = 0.08$ this is 0.980 vs. 0.830 vs. 0.362, and by $\lambda = 0.24$ the gap has widened in relative terms to 0.715 vs. 0.150 vs. 0.009. This tracks the ordering of true coefficient magnitude ($\beta_3 = 0.30 > \beta_4 = 0.15 > \beta_5 = \dots = \beta_{10} = 0$): a larger true coefficient produces a larger, more reliably detectable in-sample correlation with Y , so a larger λ is required to push its estimate below the soft-threshold. The noise block’s average frequency is not negligible at the smallest penalty (0.362) — with ten covariates and $n = 120$, a few spurious correlations are expected to survive a weak penalty — but it decays roughly forty-fold from $\lambda = 0.08$ to $\lambda = 0.24$, confirming that penalty strength is doing most of the work of separating signal from noise once λ is large enough.

The (X_1, X_2) results show why a fitted zero does not establish that the population coefficient is zero, and a nonzero fit does not establish unique importance. In a replicate where lasso selects only X_2 , the true $\beta_1 = 0.15$ is identical to β_2 — X_1 is just as related to Y marginally, but was dropped only because X_2 already absorbs nearly the same information. Symmetrically, the nonzero $\hat{\beta}_2$ in that replicate is not evidence that X_2 uniquely drives Y : it stands in for a shared effect that could equally have been attributed to X_1 in a different sample, as the near-equal magnitudes of **only** X_1 (0.465 at $\lambda = 0.08$) and **only** X_2 (0.465) in Table 3 confirm. Under high correlation, lasso’s selection pattern reflects which member of a redundant group happened to be picked in that sample — not which variable is structurally “the” important one.

Question 2: Elastic net with an equal penalty mix

Answer

Lasso: all 800 fits converged: **True**. Elastic net: all 800 fits converged: **True**.

Table 5: Elastic net ($\alpha = 0.5$) selection frequency by covariate and λ .

	0.08	0.12	0.18	0.24
--	------	------	------	------

Part aTable 5: Elastic net ($\alpha = 0.5$) selection frequency by covariate and λ .

	0.08	0.12	0.18	0.24
$X_{\{1\}}$	0.970	0.985	0.960	0.935
$X_{\{2\}}$	0.970	0.980	0.965	0.940
$X_{\{3\}}$	1.000	0.985	0.980	0.950
$X_{\{4\}}$	0.935	0.880	0.775	0.660
$X_{\{5\}}$	0.610	0.505	0.280	0.170
$X_{\{6\}}$	0.690	0.505	0.325	0.195
$X_{\{7\}}$	0.705	0.530	0.360	0.230
$X_{\{8\}}$	0.640	0.505	0.315	0.180
$X_{\{9\}}$	0.630	0.450	0.275	0.135
$X_{\{10\}}$	0.660	0.460	0.280	0.180

Table 6: Elastic net minus lasso selection frequency, same datasets ($n = 120$).

	0.08	0.12	0.18	0.24
$X_{\{1\}}$	+0.445	+0.475	+0.485	+0.555
$X_{\{2\}}$	+0.445	+0.505	+0.530	+0.600
$X_{\{3\}}$	+0.020	+0.035	+0.100	+0.235
$X_{\{4\}}$	+0.105	+0.220	+0.410	+0.510
$X_{\{5\}}$	+0.255	+0.335	+0.245	+0.170
$X_{\{6\}}$	+0.320	+0.310	+0.280	+0.185
$X_{\{7\}}$	+0.285	+0.300	+0.290	+0.225
$X_{\{8\}}$	+0.230	+0.325	+0.270	+0.170
$X_{\{9\}}$	+0.325	+0.310	+0.235	+0.120
$X_{\{10\}}$	+0.345	+0.290	+0.230	+0.165

The elastic-net selection frequencies (Table 5, Figure 1) are higher than lasso’s for *every* covariate at *every* λ (Table 6 is entirely positive). The largest gains, by far, belong to X_1 and X_2 : their selection frequencies jump from roughly 0.34–0.53 under lasso (Question 1) to 0.93–0.99 under elastic net, a gain that *grows* with λ (from about +0.44 at $\lambda = 0.08$ to about +0.56–0.60 at $\lambda = 0.24$). X_4 also gains substantially, and its gain keeps growing across the whole grid — from +0.105 at $\lambda = 0.08$ up to +0.510 at $\lambda = 0.24$, the largest single gain in the table. X_3 , already near-saturated under lasso, gains only modestly until the largest penalty (+0.020 at $\lambda = 0.08$ rising to +0.235 at $\lambda = 0.24$). The noise covariates $X_5, \dots, X_{10}$ gain more moderately in absolute terms (+0.12 to +0.35) but represent the biggest *relative* increase, since their lasso baseline was already small. In short: the covariates that change the most, and in the clearest direction (upward), are the near-collinear pair X_1, X_2 — precisely the pair the ℓ_2 component was designed to keep together.

Part bTable 7: Elastic net: joint and marginal selection outcomes for (X_1, X_2) .

λ	Only X_1	Only X_2	Both	Neither	Marg. X_1	Marg. X_2
0.08	0.025	0.025	0.945	0.005	0.970	0.970
0.12	0.015	0.010	0.970	0.005	0.985	0.980
0.18	0.020	0.025	0.940	0.015	0.960	0.965
0.24	0.010	0.015	0.925	0.050	0.935	0.940

Under elastic net, the “both selected” outcome dominates completely: it is 0.945 at $\lambda = 0.08$ and stays at 0.925–0.970 across the whole grid, compared to only 0.030–0.060 under lasso (Table 3 from Question 1). “Only X_1 ” and “only X_2 ” — the outcomes that dominated under lasso — are now rare (each at or under 0.025 at every λ), and “neither” stays negligible except at the largest penalty (0.050 at $\lambda = 0.24$, versus 0.005–0.015 elsewhere). So yes: the two covariates are now properly selected *together* almost every time, essentially resolving the “arbitrary single-variable pick” behavior seen under lasso.

This is exactly the elastic net’s designed grouping effect. Lasso’s penalty is $\lambda \|\beta\|_1 = \lambda \sum_j |\beta_j|$, which is invariant to how a fixed total coefficient budget is split across two identical, highly correlated columns — putting all the weight on one variable costs exactly the same, in ℓ_1 norm, as putting all the weight on the other, or anything in between, so the optimizer has no reason to prefer splitting the coefficient. Adding a squared ℓ_2 term, $\frac{1-\alpha}{2} \lambda \|\beta\|_2^2 = \frac{1-\alpha}{2} \lambda \sum_j \beta_j^2$, breaks that degeneracy: because the square is a strictly convex function, splitting a fixed total coefficient c evenly between two identical, perfectly (or near-perfectly) correlated predictors, $\beta_1 = \beta_2 = c/2$, gives a strictly smaller ℓ_2 penalty ($2 \cdot (c/2)^2 = c^2/2$) than concentrating it on one, $\beta_1 = c, \beta_2 = 0$ (penalty c^2). The elastic net therefore actively prefers to spread the coefficient across both near-duplicate covariates rather than arbitrarily picking one, which is why (X_1, X_2) are selected as a pair almost every time rather than individually.

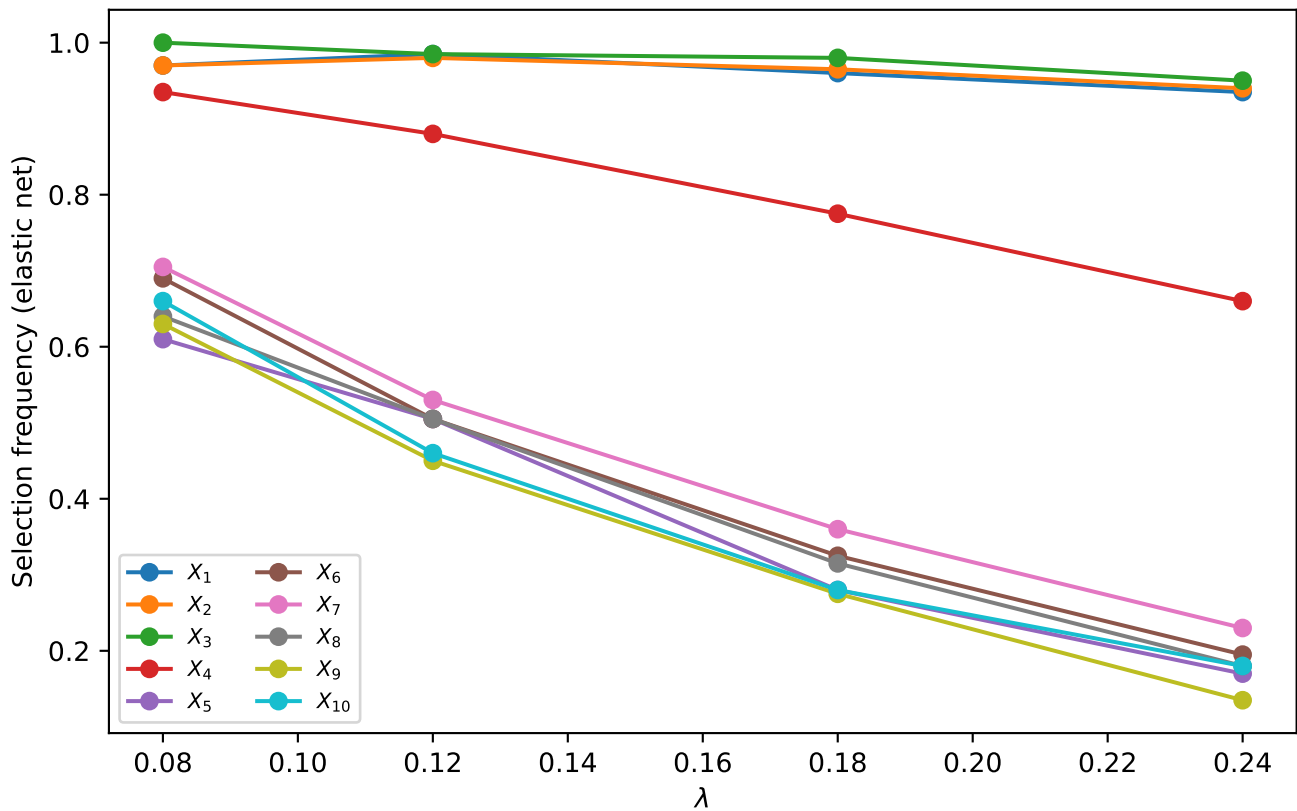


Figure 1: Elastic net selection frequency vs. λ , by covariate.

Part c

Table 8: Noise-block ($X_5, \dots, X_{10}$) average selection frequency: lasso vs. elastic net.

λ	Lasso avg($X_5, \dots, X_{10}$)	ENet avg($X_5, \dots, X_{10}$)
0.08	0.362	0.656
0.12	0.181	0.492
0.18	0.048	0.306
0.24	0.009	0.182

The noise covariates are still screened out as λ increases under elastic net — their average frequency falls monotonically from 0.656 at $\lambda = 0.08$ to 0.182 at $\lambda = 0.24$ — but at every λ this is substantially higher than the lasso baseline (0.362 $\rightarrow$ 0.009 over the same grid), roughly 1.8 $\times$ to 20 $\times$ as large. This is a direct consequence of the mixing: at a given numerical λ , sklearn’s `ElasticNet` objective applies only $\alpha\lambda = 0.5\lambda$ of ℓ_1 weight (the rest of the penalty budget goes to the ℓ_2 term), so the effective ℓ_1 threshold that drives small coefficients exactly to zero is roughly half as strong as lasso’s at the same λ . A weaker ℓ_1 threshold means noise coefficients that would have been soft-thresholded to zero under lasso survive under elastic net, which is why the noise block is screened out far more slowly.

Combining parts (b) and (c): the elastic net achieves near-complete joint selection of (X_1, X_2) — “both” ≥ 0.925 — across the *entire* grid $\lambda \in \{0.08, 0.12, 0.18, 0.24\}$, but noise-block selection frequency only drops to a comparably low range (0.182, still well above lasso’s 0.009 at the same λ) at the largest penalty, $\lambda = 0.24$. So $\lambda = 0.24$ is the penalty in this grid that comes closest to simultaneously keeping the correlated pair together (both selected 92.5% of the time) while most effectively screening out the noise covariates (down to 18.2% average selection, versus 31–66% at the smaller penalties) — though even at $\lambda = 0.24$ noise selection remains well above the lasso benchmark, reflecting the inherent trade-off of the weaker ℓ_1 threshold built into the equal-mix elastic net.

Question 3: Coordinate descent and the lasso path

Answer

```
def soft(a, lam):
    return np.sign(a) * max(abs(a) - lam, 0.0)

def coord_descent(Xs, Yc, lam, beta_init, tol=1e-8, max_iter=100000):
    beta = beta_init.copy()
    n, p = Xs.shape
    for it in range(1, max_iter + 1):
```

```

max_change = 0.0
for j in range(p_):
    # residual leaving covariate j out of the current fit
    resid_j = Yc - Xs @ beta + Xs[:, j] * beta[j]
    a_j = np.mean(Xs[:, j] * resid_j)
    new_beta_j = soft(a_j, lam)
    max_change = max(max_change, abs(new_beta_j - beta[j]))
    beta[j] = new_beta_j
if max_change < tol:
    return beta, it, True
return beta, max_iter, False

beta_path = np.zeros((len(lambdas), p))
converged_flags = []
iters_used = []

beta_curr = np.zeros(p) # warm start: begin the grid at beta = 0
for i, lam in enumerate(lambdas):
    beta_curr, iters, conv = coord_descent(Xs, Yc, lam, beta_curr) # warm start from previous lambda
    beta_path[i, :] = beta_curr
    converged_flags.append(conv)
    iters_used.append(iters)

all_converged = all(converged_flags)
lambda_max = np.max(np.abs(Xs.T @ Yc / n))
beta_ols = np.linalg.lstsq(Xs, Yc, rcond=None)[0]

```

Part a

Table 9: Coordinate-descent lasso path with warm starts.

	$\hat{\beta}_1$	$\hat{\beta}_2$	$\hat{\beta}_3$
λ			
2.50	0.00000	-0.00000	0.0
2.00	0.00000	-0.12703	0.0
1.50	0.00000	-0.62703	0.0
1.00	0.00000	-1.12703	0.0
0.60	0.32225	-1.56571	0.0
0.40	0.54954	-1.79300	0.0
0.25	0.72000	-1.96346	0.0
0.15	0.83364	-2.07710	0.0
0.08	0.91319	-2.15665	0.0
0.03	0.97001	-2.21347	0.0

All 10 fits converged (max. coefficient change $< 10^{-8}$): **True**. Iterations per grid value: [1, 2, 2, 2, 6, 6, 6, 6, 6, 6]. $\lambda_{\max} = \|\frac{1}{n}X^T\tilde{y}\|_{\infty} = 2.1270$.

Grid values at or above $\lambda_{\max}$: [2.5]. All slopes zero there: **True**.

All ten fits converged, and convergence was fast throughout — never more than 6 full cycles, and in some cases just 1–2, because each fit was warm-started from the coefficients found at the adjacent (larger) λ . Consistent with the theory, $\lambda_{\max} \approx 2.127$, and at the one grid value at or above it, $\lambda = 2.5$, all slopes are exactly zero. At the very next grid value, $\lambda = 2.0 < \lambda_{\max}$, $\hat{\beta}_2$ has already become nonzero (−0.127) — exactly what $\lambda_{\max}$ is supposed to mark: the largest penalty at which any coefficient first enters the model.

Part b

As λ decreases from 2.5 toward 0.03 (equivalently, moving toward more negative $\log \lambda$, since $\log(2.5) \approx 0.92$ and $\log(0.03) \approx -3.51$), the covariates enter the model in this order: X_2 first (nonzero already at $\lambda = 2.0$), then X_1 (nonzero starting at $\lambda = 0.6$), while X_3 never enters at any grid value. This order matches the sizes of the true coefficients: $|\beta_2| = 2.0 > |\beta_1| = 1.0 > |\beta_3| = 0$. Because the covariates are independent standard normals here (no near-collinearity as in Q1–Q2), the score $a_j = \frac{1}{n}x_j^T\tilde{y}$ that drives the soft-threshold update is, in expectation, proportional to the true coefficient β_j ; the covariate with the largest true effect produces the largest score and so is the first to exceed the shrinking penalty λ and enter the model, with smaller true effects entering only once λ has fallen further, and a zero true effect never entering (at least not from real signal — see part c).

Part c

Table 10: Lasso fit at the smallest grid penalty vs. least squares on the same standardized data.

	Lasso at $\lambda=0.03$	OLS on standardized X
$\hat{\beta}_1$	0.97001	1.00755
$\hat{\beta}_2$	-2.21347	-2.24449

Table 10: Lasso fit at the smallest grid penalty vs. least squares on the same standardized data.

	Lasso at $\lambda=0.03$	OLS on standardized X
$\hat{\beta}_3$	0.00000	0.02833

X_3 has a true population coefficient of zero, and in this run it stays *exactly* zero at every grid value, including the smallest penalty $\lambda = 0.03$ — it never enters the path. This happens because X_3 's score $a_3 = \frac{1}{n}x_3^\top \tilde{y}$ never exceeds 0.03 in absolute value for this particular sample, so the soft-threshold $S(a_3, 0.03) = \text{sign}(a_3)(|a_3| - 0.03)_+$ is zero at every λ tried.

Comparing the lasso fit at $\lambda = 0.03$ with ordinary least squares on the same standardized design: $\hat{\beta}_1 = 0.970$ (lasso) vs. 1.008 (OLS), $\hat{\beta}_2 = -2.213$ (lasso) vs. -2.244 (OLS), and $\hat{\beta}_3 = 0$ (lasso) vs. 0.028 (OLS, nonzero because OLS applies no penalty at all). The two fits are close but not identical for two related reasons. First, $\lambda = 0.03$ is small but strictly positive, so lasso still applies a small amount of soft-thresholding shrinkage to every coordinate — pulling $\hat{\beta}_1$ and $\hat{\beta}_2$ slightly toward zero relative to their unpenalized OLS values — which is why lasso's magnitudes are both slightly smaller than OLS's in absolute value. Second, and more visibly for X_3 , lasso's soft-threshold sets any coefficient whose score falls below λ in absolute value to *exactly* zero, while OLS has no such mechanism and instead returns whatever small nonzero value (0.028) best fits the sampling noise in that column. As $\lambda \rightarrow 0$ the coordinate descent solution converges to the OLS solution exactly (since the penalty vanishes), so the residual gap seen here at $\lambda = 0.03$ reflects the last bit of shrinkage and thresholding still present at that grid's smallest value.

Question 4: Tuning and comparing penalized regressions for diabetes prediction

Answer

Setup notes

Because `diabetes.csv` carries no explicit row identifier, the join to `diabetes-split-folds.csv` is done by row position: a sequential `row_id` (1, ..., 442) is attached to `diabetes.csv` in file order and matched against the `row_id` column already present in the split file. This assumption is checked directly: `row_id` in the split file is exactly the sequence 1, ..., 442, every training row carries a fold label 1, ..., 10, and every test row's fold label is missing — both assertions above pass, supporting the row-order join.

Cross-validation results

Table 11: Selected penalty and cross-validation error for each method (10-fold CV on the supplied folds).

Method	$\alpha_{\min}$	CV MSE	CV SE
Lasso	0.1000	3071.2469	240.3527
Ridge	0.5012	3073.4858	242.1468
Elastic net ($\alpha = 0.5$)	0.0032	3073.4487	242.1001

The three tuned fits are nearly indistinguishable: minimum cross-validation MSEs of about 3071.2 (lasso), 3073.5 (ridge), and 3073.4 (elastic net), each with fold-based standard errors around 240. The gap between the three methods is small compared with the displayed fold-to-fold variability — this descriptive comparison nominates a winner for this particular fold assignment, but it does not establish that any one method is truly superior for this prediction problem.

Selected model and final test evaluation

Table 12: Refit of the selected model on all training rows; test MSE evaluated once.

Method	α	Nonzero coefs	Validation MSE	Training MSE	Test MSE
Lasso	0.1	9	3071.2469	2908.3586	2732.4104

Selected covariates: `age`, `sex`, `bmi`, `bp`, `s1`, `s2`, `s4`, `s5`, `s6`.

Dropped: `s3`.

Lasso is the nominal winner, at a penalty of $\alpha \approx 0.10$. The selected fit keeps nine of the ten covariates (all but `s3`), with training MSE about 2908.4 and a final test MSE of about 2732.4 — evaluated exactly once, after the model and its penalty were already fixed by the cross-validation comparison, so this is an honest assessment of the complete selected procedure. The test error happens to fall slightly below the validation error here, but this ordering is itself a feature of this particular test split and could reverse under a different one. Because the comparison across lasso, ridge, and elastic net used the same ten folds throughout and the test rows were untouched until the method and penalty were both fixed, a different fold assignment could plausibly have nominated ridge or elastic net instead without changing anything about the validity of the procedure — the cross-validation table and plots, not the identity of the winner, are what justify the selection.

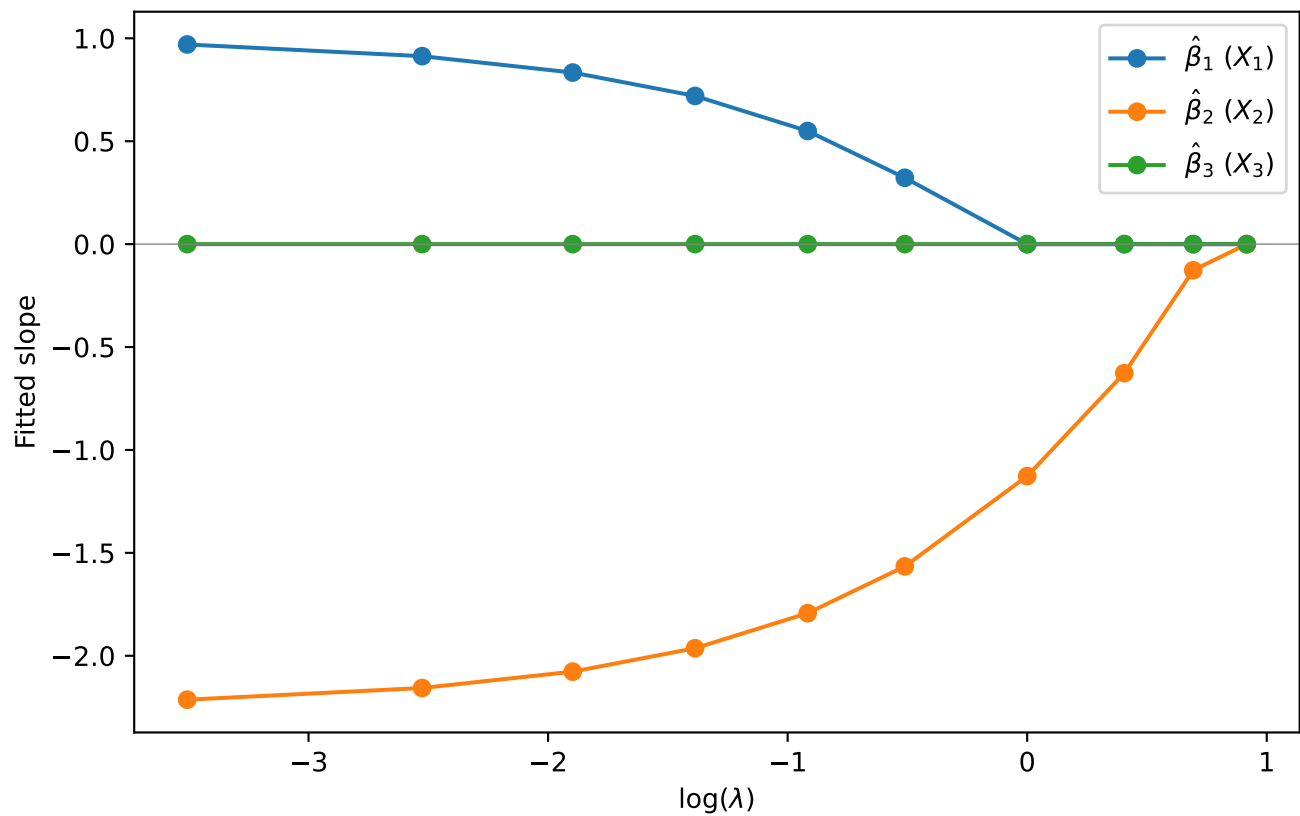


Figure 2: Lasso coefficient path vs. $\log(\lambda)$, coordinate descent with warm starts.

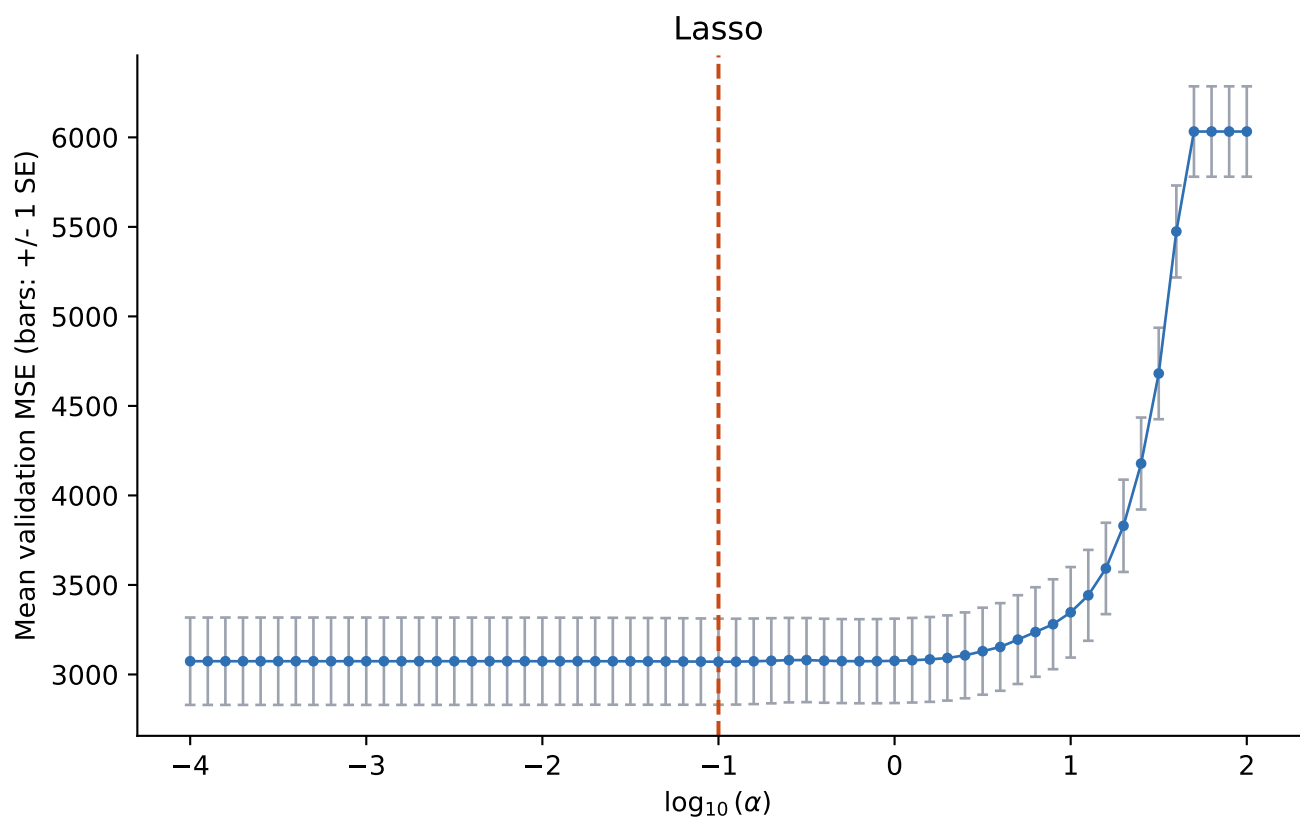


Figure 3: Cross-validation curve, Lasso: mean validation MSE vs. $\log_{10}(\alpha)$, with ± 1 SE bars. Dashed line marks the selected penalty.

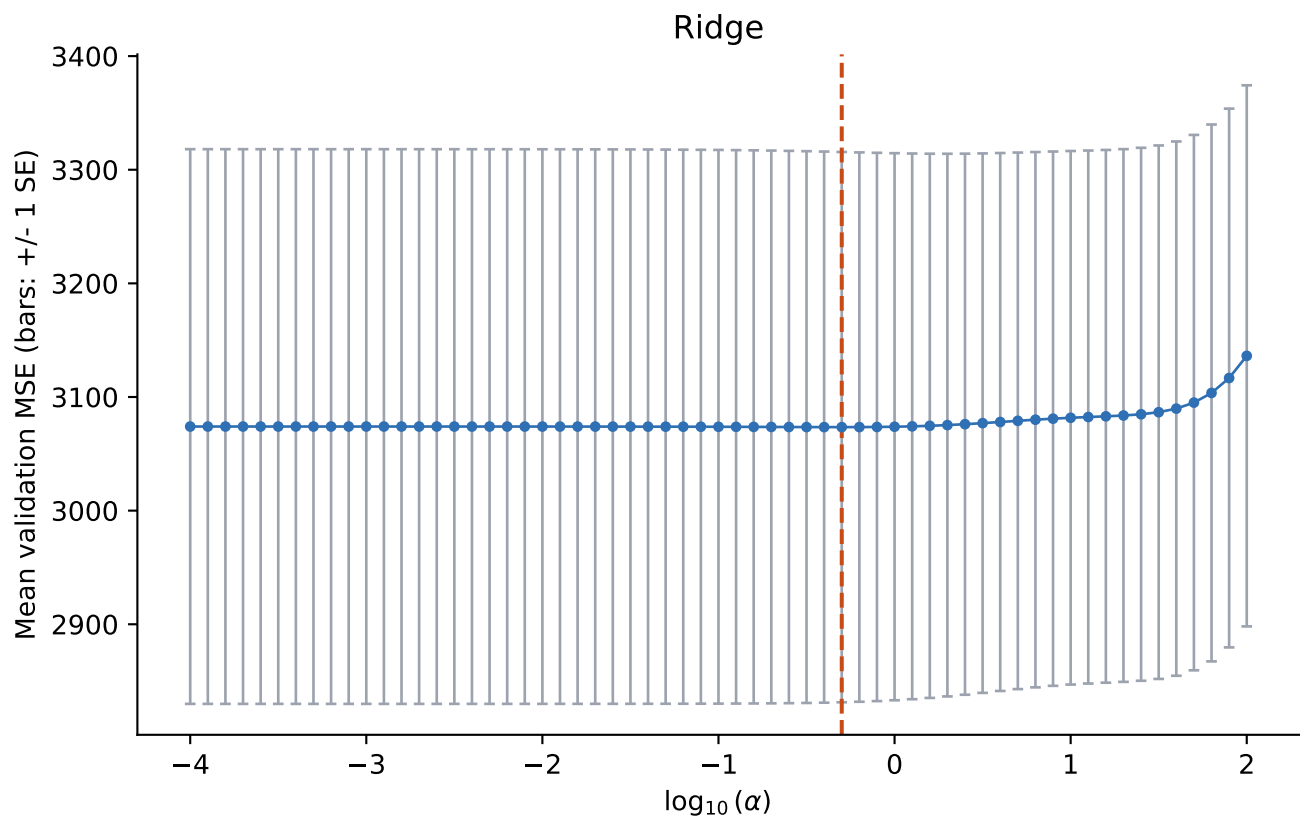


Figure 4: Cross-validation curve, Ridge.

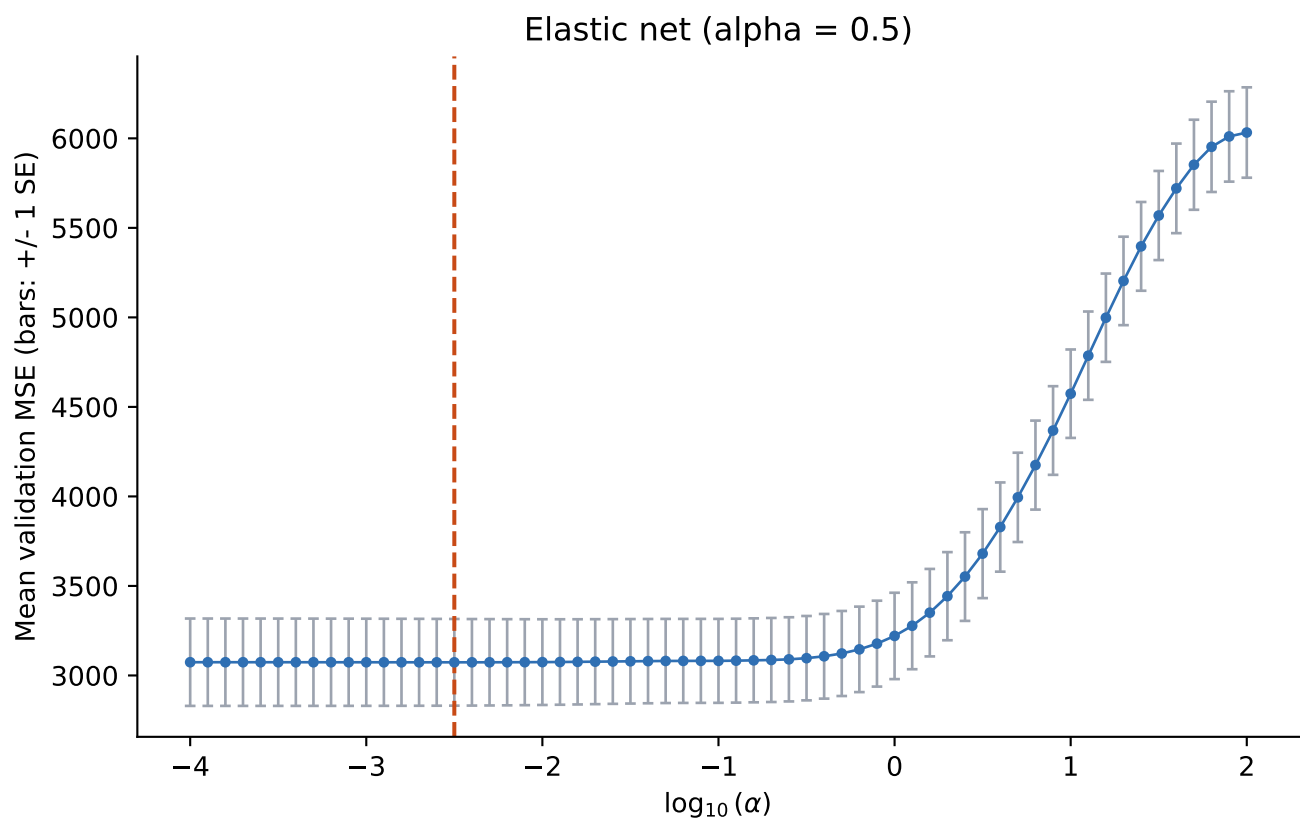


Figure 5: Cross-validation curve, Elastic net ($\alpha = 0.5$ mixing).